



IUT Nancy Charlemagne
Université de Lorraine
2 ter boulevard Charlemagne
BP55227
54052 Nancy Cedex
Département informatique

DOOM Project - Moteur de Jeu 2.5D et Multijoueur P2P

Tuteur : M. Vincent Thomas

Rapport rédigé par:

Cyprian Chailan, Geoffrey Gros, Ilyès Lounici, Marcelin Ray

BUT Informatique

Année scolaire : 2025-2026

Table des matières

1. Introduction	3
1.1. Objet du document	3
1.2. Présentation du projet et contexte technologique	3
2. Analyse	4
2.1. Découpage fonctionnel du projet	4
2.2. Modélisation UML et architecture des données	5
2.3. Évolutions par rapport à l'étude préalable de décembre	5
3. Réalisation Technique	6
3.1. Architecture logicielle et rendu de base	6
3.2. Moteur Graphique : L'algorithme de Raycasting et la méthode DDA	6
3.3. Moteur Graphique Avancé : Le partitionnement binaire de l'espace (BSP)	8
3.3.1 Construction récursive de l'Arbre BSP	8
3.3.2 Algorithme du Peintre inversé et Traversée de l'Arbre BSP pour le Rendu	9
3.3.3 Comparaison avec l'implémentation de DOOM (1993)	11
3.4. Système Multijoueur P2P : Synchronisation et Sockets TCP	12
3.5. Intelligence Artificielle et déplacement des monstres	13
3.5.1. Pathfinding : Algorithmes A*, RRT et RRT*	13
3.5.2. Simulation de mouvement et comportement cinématique : Les Steering Behaviors	17
3.6. Tests de validation, débogage et difficultés rencontrées	19
3.6.1. Précision mathématique du BSP et Epsilon	19
3.6.2. Réseau P2P	19
3.6.3. Assurance qualité et Tests automatisés	19
4. Planning de déroulement du projet durant les 3 itérations de ce semestre	20
5. Répartition du travail entre les étudiants	20
6. Présentation d'un élément original et points de fierté	21
7. Objectifs à atteindre pour la fin du projet	22
8. Planning pour les trois itérations restantes	23
9. Conclusion	23

1. Introduction

1.1. Objet du document

Ce document constitue le rapport d'avancement exhaustif de fin de semestre pour le projet tutoré intitulé "DOOM Like". Il détaille de manière approfondie le travail de conception, d'analyse et d'implémentation logicielle réalisé durant les trois premières itérations de développement, couvrant la période de mi-octobre 2025 à mi-février 2026.

L'objectif de ce rapport dépasse le simple état des lieux temporel. Il s'agit de fournir une analyse critique, mathématique et technique des choix architecturaux mis en œuvre par l'équipe de développement. Le document expose les défis algorithmiques relevés pour la construction d'un moteur graphique programmé de zéro, la mise en réseau sans serveur centralisé, et la création d'une intelligence artificielle adaptative capable de naviguer dans des géométries complexes. Enfin, il établit la feuille de route stratégique et opérationnelle pour la seconde moitié du projet, axée sur la fusion des différentes briques technologiques développées en parallèle au sein du dépôt de code source.

1.2. Présentation du projet et contexte technologique

Le projet consiste en la réalisation complète d'un moteur de jeu vidéo de type jeu de tir à la première personne (FPS), s'inspirant fortement de l'architecture historique et des innovations techniques du célèbre jeu *DOOM* sorti par id Software en 1993. Au-delà de l'aspect purement ludique, ce travail revêt une dimension pédagogique et technique majeure. Il impose l'implémentation de concepts complexes relevant de l'informatique graphique, des mathématiques appliquées (algèbre linéaire et géométrie spatiale) et des systèmes distribués, sans recourir à des moteurs de jeu commerciaux modernes tels que Unity ou Unreal Engine. Cette contrainte d'implémentation "de zéro" en langage Java garantit une compréhension complète des algorithmes sous-jacents qui motorisent les applications 3D en temps réel.

Pour satisfaire ces ambitions, on s'est imposé un cahier des charges ayant plusieurs contraintes techniques de haut niveau. La première réside dans le développement d'un moteur de rendu hybride. Nous avons ainsi dû explorer une double approche graphique : une base en *Raycasting* s'appuyant sur des grilles orthogonales bidimensionnelles, et une évolution vers un moteur basé sur des arbres de partitionnement binaire de l'espace (BSP ou Binary Space Partitioning). Cette seconde approche, historiquement révolutionnaire lors de sa conception par John Carmack, permet le rendu de géométries complexes, de murs non orthogonaux et de secteurs asymétriques avec une efficacité redoutable.

La deuxième contrainte majeure concerne l'architecture réseau. Contrairement à une approche Client-Serveur traditionnelle où un serveur dédié fait autorité absolue sur l'état de la simulation, nous nous sommes imposé un réseau purement Peer-to-Peer (P2P). Dans ce modèle, chaque instance du jeu doit gérer sa propre simulation locale tout en se synchronisant de manière décentralisée avec l'ensemble des autres pairs.

Enfin, le projet intègre une dimension “d'Intelligence Artificielle”. Le comportement des entités non-joueurs (les monstres) est régi par des algorithmes de recherche de chemin (pathfinding). L'implémentation couvre à la fois l'algorithme déterministe A* pour les espaces discrétisés (les grilles du Raycasting), et les algorithmes probabilistes \$RRT\$ (Rapidly-exploring Random Tree) et RRT* pour les environnements spatiaux continus et complexes propres au rendu BSP. À l'heure actuelle, le développement s'articule autour de deux branches distinctes sur le dépôt de contrôle de version GitHub : la branche `actionJoueur`, dédiée à la boucle de jeu, à l'IA et à la communication réseau, et la branche, dédiée à l'architecture mathématique du moteur de rendu spatial. L'unification de ces deux branches constitue l'enjeu majeur des prochaines itérations.

2. Analyse

2.1. Découpage fonctionnel du projet

Afin de maîtriser la complexité due à la création d'un moteur de jeu complet et de faciliter le travail collaboratif en parallèle, l'architecture logicielle a été décomposée en plusieurs modules fonctionnels. Cette séparation des responsabilités garantit que les modifications algorithmiques apportées à l'affichage n'impactent pas la logique de simulation physique ou les communications réseau.

Le premier pilier est le Module Réseau (encapsulé dans le package Java Réseau). Ce module constitue la colonne vertébrale des interactions multijoueurs. En l'absence de serveur dédié, ce module repose sur un modèle d'architecture distribuée. Il a la responsabilité exclusive d'écouter les connexions entrantes, d'initier des requêtes de connexion vers de nouveaux pairs découverts, et de sérialiser ainsi que désérialiser les paquets de données relatifs à la position spatiale et aux actions des joueurs. Il assure la propagation de l'état du jeu selon une topologie de graphe complètement connecté, où chaque participant maintient une connexion avec tous les autres participants pour échanger des coordonnées en temps réel.

Le deuxième pilier est le Module Moteur Graphique (package `moteur_graphique`). C'est le composant responsable de la synthèse d'images en temps réel. Sa fonction est de transformer les structures de données abstraites représentant le monde virtuel (tableaux de segments de murs ou tableaux binaires) en un affichage 2.5 dimensions projeté à l'écran du joueur. Ce module a considérablement évolué depuis la phase de conception initiale pour supporter deux stratégies de rendu distinctes implémentant une même interface de programmation. La première utilise une approche volumétrique simplifiée basée sur le Raycasting sur grille, exploitant l'algorithme DDA. La seconde opère dans un espace vectoriel continu via le découpage spatial récursif (BSP).

Le troisième pilier est le Module Entités et Intelligence Artificielle (packages `entite` et `monstre`). Ce module orchestre la logique métier fondamentale et la simulation physique. Il gère la détection des collisions, le cycle de vie des joueurs locaux, la mise à jour des représentations des joueurs distants (avatars réseau recevant leurs coordonnées via le module réseau), et le comportement des ennemis autonomes. Il encapsule de manière isolée les algorithmes mathématiques d'optimisation de trajectoire (A*, RRT*, ...) qui

permettent aux entités contrôlées par l'ordinateur de naviguer dans l'environnement virtuel, de contourner les obstacles architecturaux et de poursuivre le joueur.

2.2. Modélisation UML et architecture des données

L'architecture logicielle s'articule autour de diagrammes de classes stricts (UML) qui définissent les relations d'héritage, d'association et de composition entre les différentes entités du code source. Cette rigueur structurelle est impérative pour un projet développé intégralement en Java, où la conception orientée objet définit les performances et la maintenabilité à long terme.

Le cœur du jeu repose sur la classe `GameLoop`. Cette classe implémente le patron de conception de la "boucle de jeu" (Game Loop), un standard dans l'industrie vidéoludique. Elle agit comme un chef d'orchestre asynchrone qui cadence l'exécution globale. À chaque itération de la boucle, elle appelle de manière séquentielle la méthode `update()` pour faire avancer la logique de simulation (physique temporelle, traitement des paquets réseau entrants, décisions des monstres) en fonction du temps écoulé (delta time), puis la méthode `render()` pour déclencher le dessin de la scène à l'écran.

Afin de gérer la dualité des technologies de rendu (Raycasting classique et partitionnement BSP), le patron de Conception *Stratégie* a été employé. Une interface logicielle nommée `GameRenderer` a été introduite, définissant les contrats et les méthodes que tout moteur d'affichage doit impérativement respecter. Les classes spécifiques de rendu implémentent cette interface de manière concrète. Cette abstraction rend le backend graphique parfaitement interchangeable à la volée, sans qu'il soit nécessaire d'altérer ou de réécrire la logique interne de la classe `GameLoop`.

Concernant spécifiquement la branche `BSPrendu github`, une modélisation structurelle dédiée a été requise pour représenter fidèlement l'arbre binaire en mémoire. La structure s'articule autour de la classe principale `ArbreBSP`, qui conserve une référence vers le nœud racine et pilote la construction récursive de l'arbre via la méthode `construireBSP()`. La représentation des nœuds suit quant à elle le patron de conception *Composite* : chaque élément instancié de la classe `NoeudBSP` représente soit un nœud intermédiaire contenant un hyperplan (un mur séparateur) et deux références vers des sous-arbres enfants (le sous-arbre gauche/devant et le sous-arbre droit/derrière), soit une feuille terminale correspondant à un espace géométrique convexe vide ou à un secteur navigable de la carte.

2.3. Évolutions par rapport à l'étude préalable de décembre

L'étude préalable réalisée et validée en novembre a servi de socle théorique, mais l'analyse pratique des contraintes de rendu a mis en évidence la nécessité de faire évoluer drastiquement le modèle de données géométriques pour supporter le partitionnement binaire de l'espace.

Initialement, dans le cadre du prototype basé sur le Raycasting, la carte (map) de l'environnement virtuel était représentée sous la forme d'un simple tableau bidimensionnel d'entiers (une matrice de taille $N \times M$), où chaque cellule possédait une valeur d'état, par exemple 0 pour un espace vide et 1 pour un mur infranchissable. Cette modélisation matricielle, bien que performante et simple à implémenter pour un algorithme d'analyse

différentielle numérique (DDA), confine l'architecture à des murs orthogonaux (strictement alignés sur les axes X et Y) formant des blocs uniformes. Elle interdit de facto toute géométrie complexe, telle que des murs obliques, des pièces non rectangulaires ou des variations subtiles d'épaisseur.

Pour s'affranchir de cette limitation bloquante, le modèle de données spatiales a été entièrement réécrit dans la branche de développement BSP. Les murs ne sont plus de simples blocs définis par leur position dans une grille, mais des segments définis vectoriellement dans un espace cartésien continu. La classe Mur est désormais caractérisée mathématiquement par deux points de l'espace, un point d'origine (x_0 , y_0) et un point de destination (x_1 , y_1), associés à des métadonnées structurelles telles que l'identifiant de la texture à appliquer.

Cette transition paradigmatique (le passage d'un espace de données discret vers un espace continu vectoriel) a provoqué des répercussions profondes sur l'ensemble de l'architecture du moteur. La détection des collisions, qui consistait auparavant à vérifier naïvement si les coordonnées tronquées du joueur correspondaient à une case de tableau non nulle, a dû être intégralement reprogrammée. Le moteur physique doit désormais utiliser des calculs d'intersection de segments de droites, des projections de vecteurs et des tests de distance point-segment pour empêcher le joueur de traverser la géométrie. Cette évolution majeure marque le principal bond technologique du projet par rapport aux spécifications établies en décembre.

3. Réalisation Technique

Cette section détaille la mise en œuvre concrète, algorithmique et mathématique des trois piliers technologiques du projet. Elle explicite les méthodes utilisées pour le rendu d'images, la synchronisation des données sur le réseau, et la prise de décision de l'intelligence artificielle.

3.1. Architecture logicielle et rendu de base

Le projet repose intégralement sur le langage de programmation Java. L'interface graphique est gérée par les bibliothèques standards de l'écosystème (Swing et AWT). Le moteur manipule directement les tampons de pixels (Framebuffers) via un Canvas ou un JPanel pour dessiner manuellement les colonnes verticales de l'écran, à la manière des contraintes matérielles logicielles des années 1990.

3.2. Moteur Graphique : L'algorithme de Raycasting et la méthode DDA

La première technologie d'affichage implémentée avec succès au sein de la branche actionJoueur est le Raycasting. Le principe fondamental de cette technique réside dans la simulation d'une perspective tridimensionnelle par la projection, depuis la position virtuelle de la caméra, d'un rayon mathématique pour chaque colonne verticale de pixels de l'écran ; ainsi, pour une résolution de 800 pixels de large, le moteur projette simultanément 800 rayons dans l'espace bidimensionnel. Cependant, une approche naïve consistant à faire avancer le rayon par de petits pas fixes serait lourde en calculs et imprécise. C'est pourquoi l'équipe a opté pour l'implémentation de l'algorithme DDA (Digital Differential Analyzer), une

méthode d'analyse différentielle optimisée garantissant la détection exacte de chaque limite de cellule de la grille traversée.

Concrètement, le processus mathématique débute par le calcul du vecteur rayon pour chaque colonne x de l'écran, obtenu via l'addition vectorielle de la direction de la vue du joueur et du plan de la caméra. Dans un second temps, l'algorithme détermine les distances différentielles (ΔDistX et ΔDistY), calculées par l'inverse de la direction du rayon ($\Delta = |1/\text{rayDir}|$), qui représentent la distance euclidienne nécessaire pour franchir une unité sur l'axe X ou Y . Par la suite, une boucle itérative rapide effectue la traversée (le "Stepping") en avançant de case en case et en incrémentant les variables d'accumulation sideDist . Ce mécanisme permet de déterminer dynamiquement si le rayon a percuté la face verticale (Est/Ouest) ou horizontale (Nord/Sud) d'un mur, une information indispensable pour l'application ultérieure des ombrages.

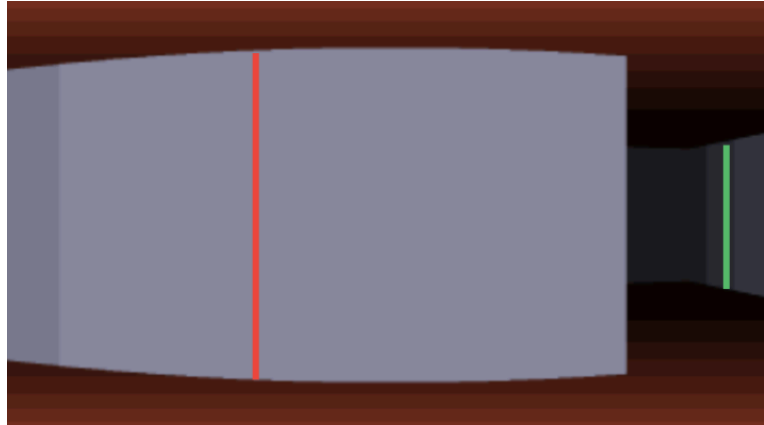
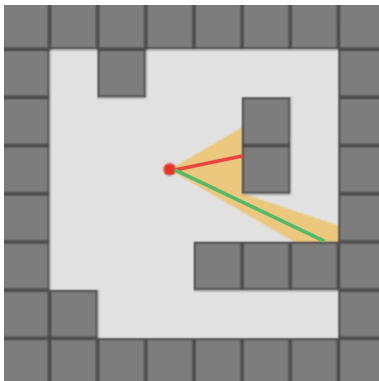
Une fois l'intersection détectée, il est impératif de corriger la distorsion spatiale connue sous le nom d'effet "Fisheye", phénomène où les murs plans paraissent courbés car les rayons latéraux parcourent une distance plus longue que ceux du centre. Pour y remédier, l'algorithme ne se base pas sur la distance euclidienne brute, mais projette cette distance orthogonalement sur le plan de la caméra pour obtenir la distance perpendiculaire perpWallDist . Enfin, cette valeur corrigée est utilisée dans une division projective ($h = \text{HauteurEcran} / \text{perpWallDist}$) pour définir la hauteur finale du mur à l'écran.

En synthèse, bien que ce moteur impose une structure spatiale basée sur une grille discrète et une géométrie strictement orthogonale, il présente l'avantage d'une gestion naturelle de l'occlusion, le rayon s'arrêtant définitivement au premier obstacle rencontré.

Caractéristiques Comparatives	Moteur Raycasting (DDA)
Structure spatiale	Grille bidimensionnelle discrète (Matrice)
Géométrie de l'environnement	Orthogonale stricte (blocs carrés alignés sur les axes)
Complexité algorithmique	Dépendante de la distance de vue (linéaire au nombre de cases)

Gestion de l'occlusion	Naturelle (un rayon s'arrête définitivement au premier mur touché)
-------------------------------	--

Visuellement :



https://en.wikipedia.org/wiki/Ray_casting

Dans cet exemple, des rayons sont envoyés depuis la position du joueur dans différentes directions. Chaque rayon avance jusqu'à rencontrer un mur. La distance entre le joueur et le point de collision est alors calculée. Cette distance permet de déterminer comment le mur sera affiché à l'écran, notamment sa hauteur, afin de créer un effet de profondeur.

3.3. Moteur Graphique Avancé : Le partitionnement binaire de l'espace (BSP)

Pour dépasser les sévères limitations orthogonales inhérentes au Raycasting et autoriser la construction d'architectures polygonales arbitraires, la branche parallèle BSPrendu a introduit l'usage des arbres de partitionnement binaire de l'espace (BSP Trees). Cette technique d'optimisation spatiale, développée à l'origine dans les années 1980 par les chercheurs H. Fuchs, Z. Kedem et B. Naylor pour des applications militaires, a été adaptée au jeu vidéo en temps réel par John Carmack pour le développement du jeu DOOM. Le développement de cette structure de données représente la réussite mathématique la plus complexe du projet tutoré.

3.3.1 Construction récursive de l'Arbre BSP

L'algorithme de construction s'exécute lors du chargement de la carte (en pré-calcul). Il prend en entrée un ensemble désordonné de segments de droite vectoriels (les murs). La méthode sélectionne un premier mur pour servir de "plan de partition" (hyperplan). Ce plan, qui s'étend à l'infini, coupe virtuellement l'ensemble de l'espace de la carte en deux demi-espaces distincts : l'espace situé "devant" la ligne directrice du mur (selon son vecteur normal), et l'espace situé "derrière".

Pour classer la position relative de tous les autres murs de la carte par rapport à cet hyperplan, l'implémentation repose sur le calcul de produits vectoriels (Cross Product), implémentés de façon optimisée dans la fonction `cotéDuPoint`. Le signe mathématique résultant du produit vectoriel entre le vecteur directeur du plan de partition et le vecteur reliant l'origine de ce plan au point à tester permet de déterminer instantanément son appartenance à l'un des demi-espaces.

La complexité surgit lorsqu'un mur de l'environnement traverse physiquement le plan de partition. Dans ce cas de figure, il est impératif pour l'intégrité de l'algorithme de scinder ce mur en deux segments distincts via la méthode `couperMur`. L'équipe a dû écrire le code calculant le point d'intersection mathématique exact entre ces deux lignes pour diviser le mur et placer chaque fragment dans le sous-arbre correspondant. Ce processus est appelé récursivement sur les deux demi-espaces obtenus jusqu'à ce que tous les murs soient intégrés à l'arbre sous forme de nœuds séparateurs ou de feuilles.

3.3.2 Algorithme du Peintre inversé et Traversée de l'Arbre BSP pour le Rendu

L'avantage structurel majeur et fondamental de l'arbre BSP réside dans sa capacité à compiler un tri implicite de la géométrie de la scène, en commençant par les murs les plus proches du joueur jusqu'aux plus éloignés (front-to-back). Cette architecture permet, lors de l'exécution, de récupérer un ordre d'affichage trié selon la profondeur depuis n'importe quel point de vue en un temps quasi linéaire.

L'enjeu par la suite est de naviguer dans ce tri efficacement. L'approche que nous avons décidé d'adopter permet de couper au plus tôt possible le rendu d'une image. Pour cela, à chaque fois que l'on stock un mur pour le dessiner, on va stocker l'intervalle en largeur qu'il va prendre sur l'écran. Ce qui nous permet par la suite de tronquer les murs qui sont derrière nos murs déjà existant, en vérifiant si leurs intervalle sur l'écran rendu est déjà ou non pris par un/des autres murs.

Ainsi, quand l'intervalle entier de l'écran est rempli, on sait qu'il est inutile de continuer le parcours de l'arbre car plus aucun mur ne pourra être affiché. Et l'on peut donc directement retourner la liste de mur à une méthode faite pour les afficher dans la fenêtre du joueur.

Ceci est bien évidemment une version très simplifiée du fonctionnement réel de notre implémentation:

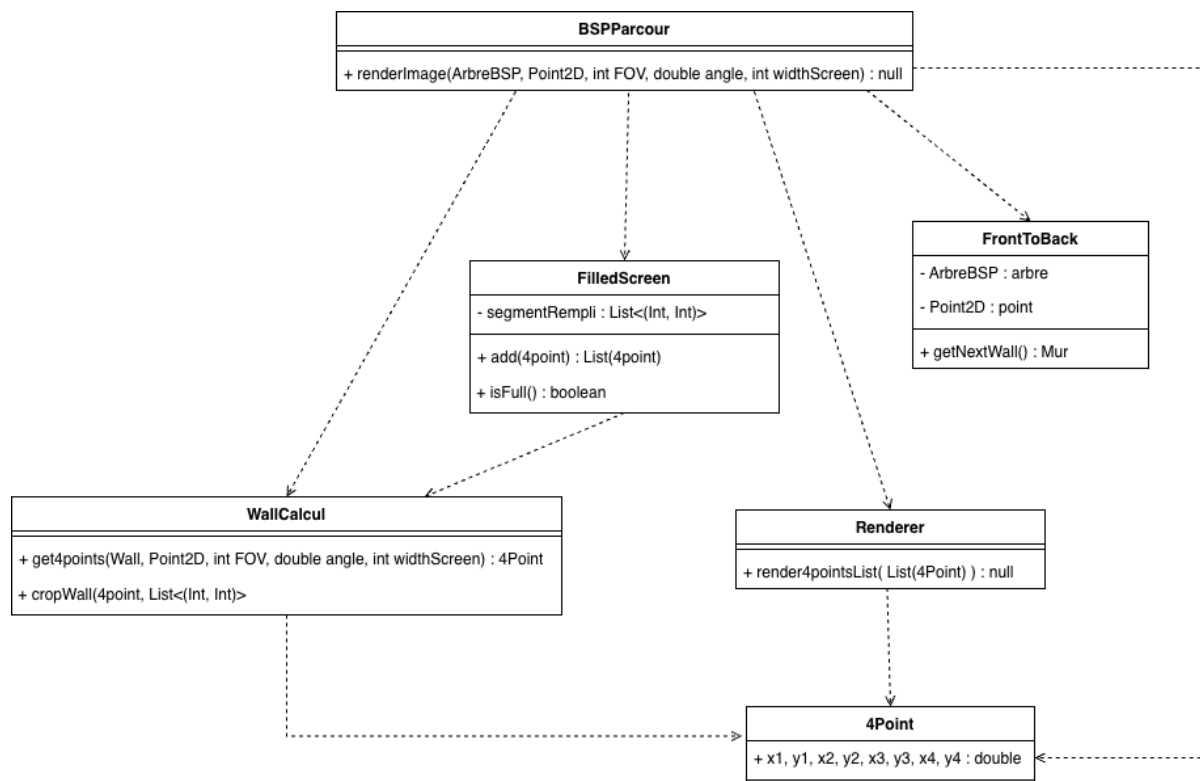


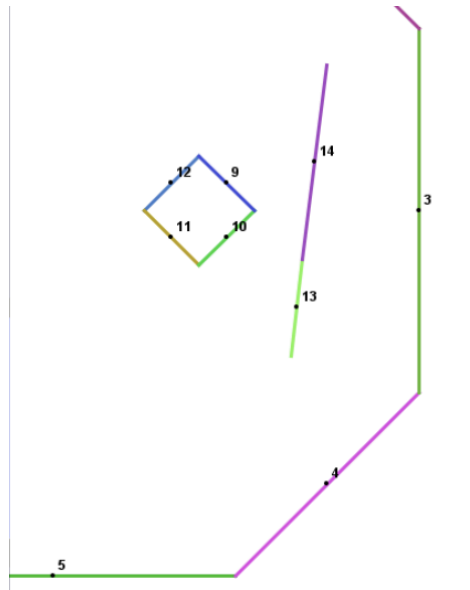
Diagramme de classe de la partie BSP Rendu

Afin de pouvoir se répartir le travail, nous avons séparé la logique et les algorithmes en plusieurs classes. Afin de pouvoir communiquer entre elles, nous avons créé une nouvelle classe QuatrePoint, permettant de stocker les 4 points qu'un mur prend, projeté sur l'écran (le premier point représente le point en haut à gauche, le deuxième en bas à gauche, ... Il suffit de remplir ces 4 points avec une méthode de AWT qui permet de remplir un polygone par la suite).

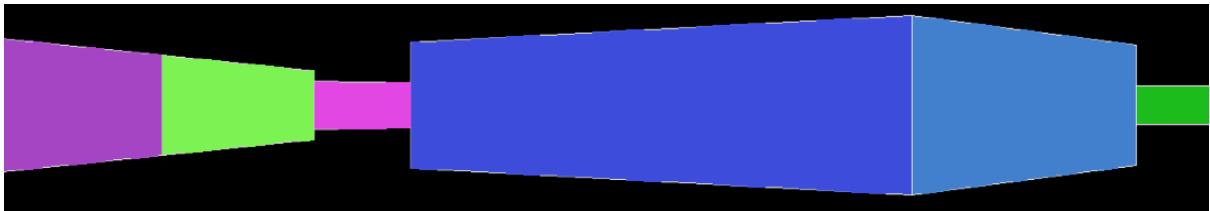
C'est nécessaire pour l'optimisation, car si on raisonne par mur, on doit calculer la projection du mur afin de pouvoir stocker l'intervalle en largeur de l'écran qu'il va utiliser, puis on devra recalculer la projection quand on va devoir tronquer si besoin le mur. Puis encore calculer la projection afin de l'afficher à la fin.

Toutes les classes fonctionnent donc ensemble, la classe BSPRendu va s'occuper de tout coordonner, du parcours jusqu'à l'affichage dans la fenêtre. FrontToBack va gérer tout le parcours de l'arbre proprement, pour pouvoir déléguer la logique de navigation dans l'arbre BSP. La classe FilledScreen va permettre de stocker les intervalles remplis de l'écran et de tronquer ou non si besoin les murs. La classe WallCalcul elle permet de projeter les points, ainsi que d'implémenter la logique de troncation. Et la classe Renderer se charge juste de remplir les polygones à la fin.

Exemple de rendu :



ArbreBSP d'une map



Rendu (les couleurs ont été ajoutées par la suite à des fins explicatives)

3.3.3 Comparaison avec l'implémentation de DOOM (1993)

Notre implémentation du BSP diffère de celle utilisée dans DOOM sur deux aspects majeurs : la structure de données et l'algorithme de parcours.

Structure de l'arbre BSP

Dans DOOM, l'arbre BSP est conçu comme une véritable décomposition spatiale hiérarchique. Les nœuds internes représentent des plans de partition abstraits qui divisent l'espace, tandis que les feuilles de l'arbre contiennent des structures appelées "subsectors" (des régions convexes qui regroupent explicitement l'ensemble des segments appartenant à une même zone de l'espace). Cette organisation permet de traiter collectivement tous les murs d'une région convexe en une seule opération.

Notre approche adopte une structure plus directe : chaque nœud de l'arbre correspond immédiatement à un segment de mur qui sert simultanément de géométrie à afficher et de plan de partition. Les zones convexes de l'espace ne sont pas stockées explicitement sous forme de subsectors, mais existent implicitement à travers la hiérarchie récursive de l'arbre. Cette simplification facilite l'implémentation au prix d'une granularité plus fine lors du parcours, puisque chaque segment doit être traité individuellement.

Algorithme de parcours et rendu

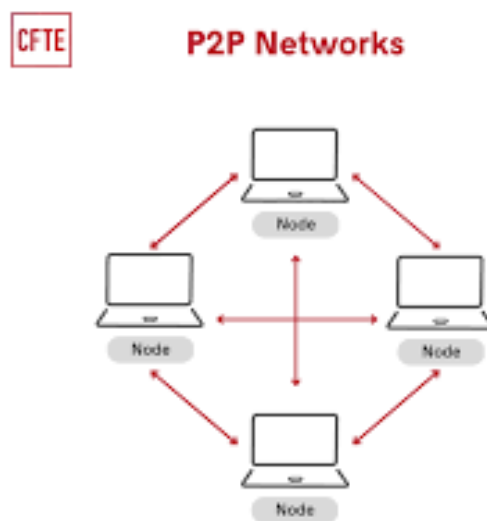
DOOM utilise l'algorithme du peintre classique avec un parcours back-to-front (du plus lointain au plus proche). L'arbre est parcouru intégralement, et les segments sont dessinés dans cet ordre, les éléments plus proches écrasant naturellement les pixels des éléments

plus éloignés. Cette approche, bien qu'exhaustive, était adaptée aux contraintes matérielles de 1993 où la gestion d'états complexes était coûteuse.

Notre implémentation adopte une stratégie front-to-back optimisée avec occlusion culling précoce. Comme décrit en section 3.3.2, nous parcourons l'arbre du plus proche au plus lointain tout en maintenant dynamiquement l'état des intervalles d'écran déjà occupés via la classe FilledScreen. Cette structure nous permet de tronquer ou d'éliminer complètement les segments occultés, et surtout d'interrompre le parcours de l'arbre dès que l'écran est entièrement rempli. Cette optimisation moderne réduit significativement le nombre de calculs de projection et d'opérations de dessin, particulièrement dans les scènes à forte profondeur.

3.4. Système Multijoueur P2P : Synchronisation et Sockets TCP

Conformément au cahier des charges où l'on s'était imposé le développement d'une architecture décentralisée, le module multijoueur repose sur une architecture purement Peer-to-Peer (P2P) à maillage complet. Bien que le code source utilise une terminologie "Client/Serveur" pour l'initialisation (notamment via la classe P2PServer qui agit comme un point d'ancrage initial), il n'y a pas de serveur dédié autoritaire une fois la partie lancée, chaque pair gère sa propre simulation.



<https://blog.cfte.education/what-is-p2p-network-blockchain/>

La gestion des flux réseau est assurée par la classe GestionConnection, qui exploite l'API des Sockets Java (`java.net.Socket` et `java.net.ServerSocket`) via le protocole TCP (Transmission Control Protocol). Bien que le protocole UDP (User Datagram Protocol) soit couramment privilégié dans les jeux d'action modernes pour la transmission rapide et sans garantie de données éphémères afin de réduire la latence, le TCP a été choisi stratégiquement pour cette première implémentation. Le TCP garantit l'intégrité, l'ordre d'arrivée et la livraison des paquets, prévenant ainsi les désynchronisations massives d'état entre les pairs lors des phases critiques de connexion.

Techniquement, chaque instance du jeu lance un noeud local (classe Serveur) qui écoute les connexions entrantes (ServerSocket) tout en maintenant une liste de connexions actives (GestionConnection) vers les autres pairs. Les échanges se font via des flux textuels (PrintWriter) plutôt que binaires pour faciliter le débogage. Lorsqu'un joueur se déplace, MainGameMultiplayer actualise la physique et transmet les coordonnées via la méthode broadcastToPeers, qui diffuse l'information à tous les sockets connectés.

Ce modèle correspond à une synchronisation d'état (State Synchronization). Contrairement au Lockstep, nous transmettons la position absolue (X, Y) calculée localement.

Pour la connexion initiale, l'absence de serveur central est palliée par un mécanisme de propagation de liste : lorsqu'un nouveau pair se connecte à un joueur existant (le seed), il reçoit immédiatement une liste de pairs connus (PEER_LIST) lui permettant d'établir ses propres connexions avec le reste du maillage.

3.5. Intelligence Artificielle et déplacement des monstres

3.5.1. Pathfinding : Algorithmes A*, RRT et RRT*

L'animation autonome des adversaires est propulsée par des modèles algorithmiques de recherche de chemin (Pathfinding) sophistiqués, l'IA devant être capable d'analyser la topologie de l'environnement en temps réel pour traquer le joueur. Initialement, en raison de la dualité du développement graphique, l'équipe a exploré deux familles d'algorithmes distinctes : une approche déterministe pour la grille et une approche probabiliste pour l'espace vectoriel. Cependant, l'évolution structurelle du projet vers un moteur à partitionnement binaire (BSP) a imposé une transition technique majeure, rendant les méthodes basées sur des grilles obsolètes pour la version finale du jeu.

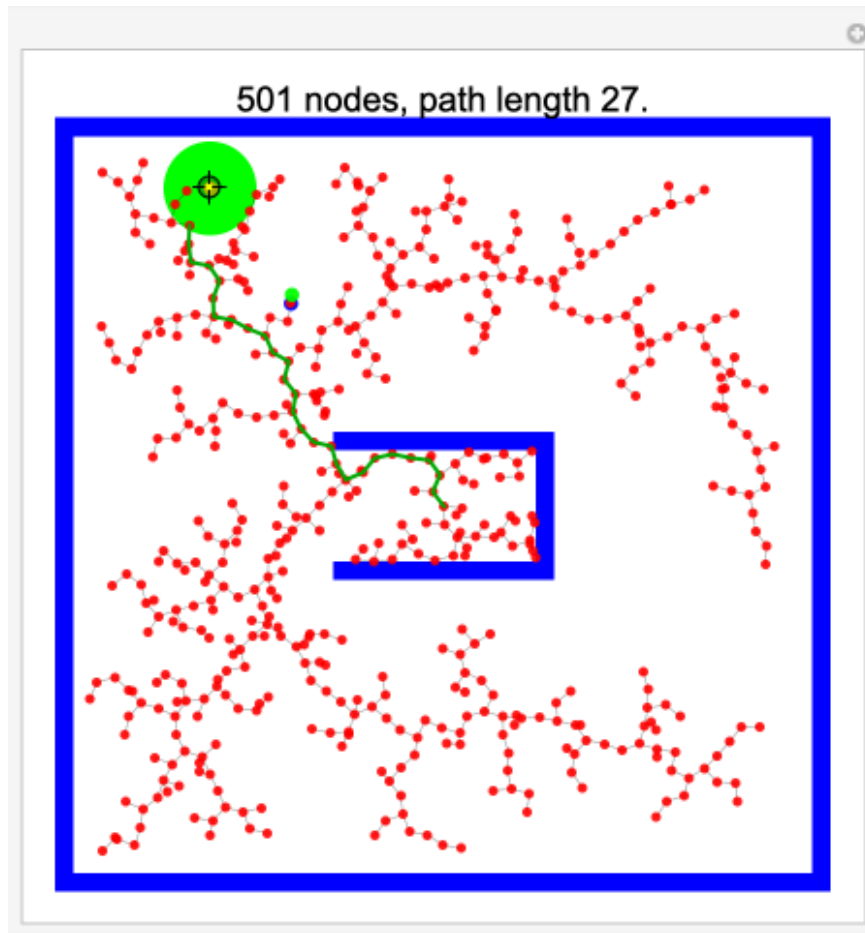
Dans la première phase consacrée au moteur Raycasting opérant sur une grille orthogonale stricte, l'algorithme A* (A-Star) a été implémenté. C'est un algorithme de parcours de graphe largement répandu garantissant la découverte du chemin le plus court. Il évalue les cellules adjacentes en minimisant de manière itérative la fonction de coût $f(n) = g(n) + h(n)$, où $g(n)$ représente le coût exact accumulé depuis le nœud de départ et $h(n)$ l'heuristique (l'estimation mathématique de la distance restante jusqu'à la cible). Dans le cadre de la grille géométrique du projet, la distance de Manhattan (définie par $D = |x1 - x2| + |y1 - y2|$) a été privilégiée comme fonction heuristique en raison de sa pertinence sur les axes orthogonaux. Toutefois, malgré son optimalité théorique sur grille, l'algorithme A* a été abandonné pour le rendu final. En effet, il souffre de problèmes d'évolutivité dans les environnements ouverts et, surtout, génère des trajectoires en "escalier" dictées par les axes de la grille, un comportement jugé inesthétique et incompatible avec la liberté de mouvement offerte par la géométrie continue du moteur BSP.

Pour pallier ces limitations et adapter l'IA à un espace vectoriel complexe, l'équipe a pivoté vers l'algorithme RRT (Rapidly-exploring Random Tree). Contrairement à l'analyse systématique des cases d'une matrice propre à A*, le RRT échantillonne l'espace continu de

manière probabiliste. Il génère un arbre en créant des points cibles aléatoires (x_{rand}) et tente de les relier au nœud existant le plus proche (x_{near}), par un segment rectiligne. Si ce segment ne croise aucun obstacle géométrique (testé via les équations d'intersection du moteur de collision), la branche est considérée comme valide et ajoutée à la structure. Ce processus stochastique se répète frénétiquement jusqu'à ce que la cible du monstre soit atteinte. Néanmoins, le RRT standard produisant par nature des chemins chaotiques et en dents de scie, souvent éloignés de la trajectoire optimale, il s'est avéré insuffisant pour garantir une expérience de jeu fluide.

C'est pourquoi le code a été finalement étendu vers la variante optimisée RRT*, qui constitue la solution retenue. Lors de l'ajout d'un nouveau nœud x_{new} , le RRT* examine tous les voisins existants dans un rayon défini. Il sélectionne le meilleur parent possible pour minimiser le coût global, puis il exécute une opération algorithmique essentielle appelée *Rewiring* (réorganisation ou recâblage). Il vérifie si ses propres voisins ne bénéficieraient pas d'un chemin globalement plus court en passant par ce nouveau nœud x_{new} . Si la condition est vérifiée, il modifie les pointeurs parents des nœuds voisins, lissant ainsi continuellement les branches de l'arbre pour atteindre une optimalité asymptotique.

Il est important de préciser que dans notre implémentation logicielle, la classe nommée RRT.java intègre directement ces optimisations propres au RRT (notamment via la méthode `optimiserConnexionNoeuds`). Par soucis de simplification du code, nous n'avons pas maintenu deux classes distinctes ; l'algorithme naïf a été remplacé par sa version optimisée dans la finale du projet



<https://demonstrations.wolfram.com/RapidlyExploringRandomTreeRRTAndRRT/>

Cette figure illustre l'algorithme RRT* planifiant un chemin optimal (ligne verte) pour s'extraire d'un obstacle concave vers une cible. Contrairement au RRT classique, cette variante optimise continuellement la structure de l'arbre pour garantir le trajet le plus court.

Pour assurer le débogage efficace de la génération procédurale et probabiliste vertigineuse de ces arbres, un outil graphique a été spécialement conçu. La classe RRTVisualisation.java dessine en temps réel la ramification arborescente (en bleu) sur un Canvas de développement indépendant, fournissant une rétroaction visuelle inestimable sur la performance de l'échantillonnage spatial et de l'évitement des murs.

Analyse Comparative	Algorithme A* (A-Star)	Algorithmes RRT et RRT*
Espace de recherche traité	Espace discret (Graphe fini ou Grille matricielle)	Espace continu (Coordonnées géométriques $\mathbb{R}^2$)

Nature de l'exploration	Approche déterministe, exhaustive et systématique	Approche probabiliste, par échantillonnage aléatoire
Garantie d'optimalité	Optimalité stricte (si l'heuristique choisie est admissible)	Optimalité asymptotique (uniquement pour la variante RRT*)
Application dans le projet	Parfaitement adapté au moteur Raycasting à grille 2D	Impératif pour la navigation fluide dans le moteur polygonal BSP

Algorithme RRT

```

function trouverChemin( entier: x,y,x2,y2 )
debut <- Noeud(x,y)
fin <- Noeud(x2,y2)

pour i allant de 1 à n faire
    n <- creerNoeud()
    n2 <- trouverPlusProche(n)
    si non collision(n, n2)
        alors n2.ajouterVoisin(n)
    si distante(n,fin)<seuil
        retourner chemin(start, fin)
retourner échec

```

Algorithme RRT*

```

function trouverChemin(entier: x,y,x2,y2)

debut <- Noeud(x,y)
fin <- Noeud(x2,y2)
debut.cout <- 0
Arbre <- {debut}

```



```

pour i allant de 1 à n faire
n <- creerNoeud()
n2 <- trouverPlusProche(n)

    si non collision(n, n2) alors
        Voisins <- trouverVoisinsDansRayon(q_new, rayon)

        //Choix du meilleur parent (minimisation du coût)
        meilleurParent <- q_proche
        coutMin <- q_proche.cout + distance(q_proche, q_new)

        pour chaque voisin dans Voisins faire
            si non collision(q_voisin, n) ET
                voisin.cout + distance(voisin, n) < coutMin alors

                meilleurParent <- voisin
                coutMin <- voisin.cout + distance(voisin, n)

        n.parent <- meilleurParent
        n.cout <- coutMin

        //Rewiring (réorganisation de l'arbre)
        pour chaque voisin dans Voisins faire
            si non collision(n, voisin) ET
                n.cout + distance(n, voisin) < voisin.cout alors

                voisin.parent <- n
                voisin.cout <- n.cout + distance(n, voisin)

        si distance(n, fin) < seuil alors
            fin.parent <- n
            retourner chemin(debut, fin)

retourner échec

```

3.5.2. Simulation de mouvement et comportement cinématique : Les Steering Behaviors

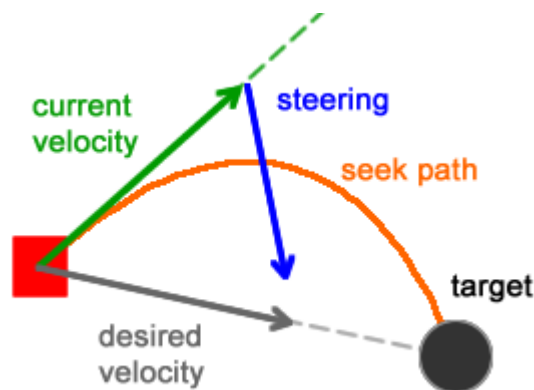
Les algorithmes de pathfinding comme RRT* sont indispensables pour trouver un chemin, mais ils ne fournissent qu'une liste de points fixes. Si nous déplaçons les monstres d'un point à l'autre en ligne droite, leurs mouvements sembleraient robotiques et sans vie. Pour corriger cela, nous avons implémenté les "Steering Behaviors" (comportements de pilotage).

Cette méthode ne positionne pas directement l'entité sur la carte. Au lieu de cela, elle simule un modèle physique en appliquant des forces sur la vitesse du monstre, exactement comme on piloterait un véhicule doté d'inertie.

Le cœur technique de cet algorithme réside dans le calcul de la force de pilotage au sein de la classe `SteeringBehavior`. Le programme détermine d'abord la "vitesse désirée" pour atteindre l'objectif. Ensuite, il lui soustrait la "vitesse actuelle" du monstre. Le vecteur résultant représente la force nécessaire pour corriger la trajectoire. Enfin, nous limitons cette force par une valeur maximale (`maxForce`) via une opération de "clamping". C'est cette contrainte mathématique qui crée l'illusion de masse et empêche les virages instantanés irréalistes. ($Force = clamp(VitesseDésirée - VitesseActuelle, ForceMax)$).

Concrètement, cette simulation repose sur l'interaction fluide entre deux comportements complémentaires. Le premier, le "Seek" (poursuite), pousse l'entité vers sa cible tout en lissant sa trajectoire pour créer des courbes naturelles plutôt que des segments brisés. Cependant, pour éviter que le monstre ne s'arrête brutalement ou n'oscille autour de sa destination finale, le comportement "Arrive" prend le relais à l'approche du but. Dès que l'entité pénètre dans un rayon de ralentissement défini, le moteur réduit progressivement la magnitude de sa vitesse, assurant un freinage en douceur jusqu'à l'arrêt complet sur l'objectif.

Enfin, cette gestion physique du mouvement nous a permis de résoudre élégamment le problème des collisions grâce à une mécanique de glissement (Wall Sliding). Au sein de la boucle de jeu, si le vecteur de mouvement calculé propulse le monstre dans un mur, nous ne stoppons pas net l'entité ; nous annulons simplement la composante de la vitesse qui la pousse contre l'obstacle. Cela permet à l'adversaire de glisser le long des parois sans se bloquer, maintenant ainsi la pression sur le joueur. Pour parfaire ce réalisme, l'orientation du sprite n'est pas instantanée mais elle est lissée mathématiquement pour s'aligner progressivement avec son mouvement, donnant une véritable impression de poids à la créature.



<https://cloud.tencent.com/developer/article/1005839>

Ce schéma illustre le mécanisme de **steering behavior** (comportement de direction), où une force de correction (**steering**) est appliquée pour modifier la trajectoire actuelle. Elle permet d'ajuster continuellement la vitesse réelle (**current velocity**) vers la vitesse souhaitée (**desired velocity**) afin d'atteindre une cible avec fluidité.

3.6. Tests de validation, débogage et difficultés rencontrées

Le cycle de développement logiciel a été éprouvé par des tests unitaire et des essais manuels, tous documentés dans l'analyse de cette première phase.

3.6.1. Précision mathématique du BSP et Epsilon

Le principal obstacle algorithmique s'est manifesté dans les routines de calcul d'intersection de segments du BSP. L'utilisation des types à virgule flottante standards pour déterminer mathématiquement si un point d'intersection appartient précisément à la surface d'un mur engendrait des comportements indéterminés critiques. Des erreurs d'arrondis infinitésimales causaient des disparitions aléatoires de murs sous certains angles de vue, ou créaient des polygones troués. La solution mathématique apportée a été l'introduction systématique d'une constante de tolérance $\epsilon = 1e - 5$ (epsilon) dans les calculs de la classe `ArbreBSP`. Cette marge d'erreur acceptée a permis de stabiliser le partitionnement spatial, évitant de classer de manière erronée un point colinéaire dans le mauvais sous-arbre lors des découpes de plans.

3.6.2. Réseau P2P

L'itération 3 a été principalement consacrée à la correction des bugs réseau identifiés lors de notre première présentation. Le débogage de cette architecture P2P a été un exercice complexe : pour reproduire les erreurs, nous devions impérativement connecter trois utilisateurs simultanément et croiser les logs de chaque machine afin de comprendre l'origine des dysfonctionnements.

Au cours de ces tests, nous avons d'abord corrigé un défaut dans l'accueil des nouveaux joueurs. Le serveur n'envoyait pas la liste des participants au nouvel arrivant, ce qui l'empêchait de partager ses propres coordonnées et donc d'exister dans la partie pour les autres pairs. Ce problème était accentué par une erreur de routage : le serveur transmettait parfois une mauvaise adresse IP aux membres déjà connectés, rendant impossible l'établissement d'une connexion stable avec le nouveau client.

Enfin, pour stabiliser l'ensemble, nous avons revu la procédure de connexion et de sortie du réseau. Nous avons mis en place un système de messages automatiques envoyés à tous les pairs dès qu'un mouvement est détecté sur le réseau. Cette modification garantit que chaque utilisateur dispose désormais de la même liste de contacts, assurant ainsi que tous les joueurs se voient correctement et en temps réel.

3.6.3. Assurance qualité et Tests automatisés

La robustesse du moteur a été éprouvée par une double méthodologie de validation. Sur le plan purement algorithmique, des suites de tests unitaires utilisant la librairie JUnit ont été rédigées. Le module `TestArbreBSP.java` s'assure de la validité mathématique stricte de la découpe des murs lors de la construction de l'arbre, tandis que le module

GestionConnectionTest.java valide le processus complexe de l'établissement des sockets et du handshake réseau. Sur le plan de la validation fonctionnelle et empirique, de multiples sessions de jeu en réseau local (LAN) ont été menées avec trois clients tournant simultanément. Ces tests d'intégration ont démontré que la réplication des mouvements spatiaux était fonctionnelle, bien qu'ils aient exposé la nécessité de mettre en œuvre des techniques de compensation et d'interpolation pour lisser la latence résiduelle perçue par les joueurs.

4. Planning de déroulement du projet durant les 3 itérations de ce semestre

La gestion de projet s'est articulée autour d'une méthodologie de développement incrémentale, divisée en trois itérations strictes rythmant la production logicielle.

- **Itération 1 (Décembre) : Fondation et Prototypage** Cette phase inaugurale a été consacrée à la mise en place de l'infrastructure de développement. L'équipe a initialisé le dépôt de versionnement Git, mis en œuvre la boucle d'exécution principale garantissant le respect du cadencement temporel, et construit un prototype visuel basique reposant sur l'algorithme orthogonal de Raycasting. En parallèle, les expérimentations préliminaires sur l'architecture distribuée ont commencé avec les premières ouvertures de Sockets TCP et les transferts rudimentaires de données entre deux instances locales du programme.
- **Itération 2 (Janvier) : Modélisation algorithmique et théorie** L'effort s'est reporté sur l'implémentation de composants mathématiques spécifiques. La programmation de l'intelligence artificielle a démarré avec le codage réussi de l'algorithme d'optimisation de chemin A* adapté à la matrice du Raycasting et l'algorithme probabiliste RRT, suivi de son évolution vers l'algorithme RRT* pour assurer une fluidité de mouvement optimale dans les corridors continus du labyrinthe. L'événement majeur de cette itération fut la finalisation de la recherche théorique approfondie sur le Partitionnement Binaire de l'Espace, qui a formellement acté la scission du code vers la nouvelle branche de développement BSPrendu.
- **Itération 3 (Février) : Maturation Technique et Complexité** La troisième itération, clôturant cette évaluation de mi-parcours, a été dédiée à la concrétisation algorithmique des concepts introduits en janvier. Le moteur de rendu BSP est devenu pleinement fonctionnel sur le plan mathématique, réussissant la coupe dynamique des murs par les hyperplans et implémentant avec succès l'Algorithme du Peintre pour la visibilité spatiale. Du côté du module réseau, le protocole P2P a été stabilisé, capable d'envoyer les coordonnées des joueurs et les états du jeu de manière fiable entre tous les participants.

5. Répartition du travail entre les étudiants

Pour chaque itération, nous travaillons en binôme afin d'assurer une répartition efficace des tâches. Cette organisation présente plusieurs avantages : la collaboration à deux permet

d'avoir un double regard sur le code, ce qui facilite les tests, la détection des erreurs et le débogage.

Lors de la phase de conception, cette approche favorise également les échanges et les débats constructifs. Confronter nos points de vue nous aide à choisir une architecture plus pertinente, à anticiper d'éventuels problèmes techniques et à améliorer la qualité globale du projet.

Enfin, cette organisation renforce la communication au sein de l'équipe. Nous effectuons régulièrement des points d'avancement afin de coordonner notre travail, ajuster les priorités et garantir la cohérence globale du projet. Cette méthode favorise également la responsabilisation de chacun, tout en maintenant une dynamique de groupe positive et collaborative.

6. Présentation d'un élément original et points de fierté

Dans le cadre d'un projet universitaire aux contraintes aussi strictes (programmation partie de zéro), surmonter les obstacles mathématiques et conceptuels constitue de véritables victoires d'ingénierie logicielle pour nous.

- **Geoffrey Gros :**

Le point dont je suis le plus fier est le travail réalisé lors de l'itération 3, en particulier la correction des bugs réseau rencontrés pendant la présentation de l'itération 1. L'identification de ces bugs a été particulièrement complexe étant donné que nous avons 2 jours entiers. Pour pouvoir les reproduire, il était nécessaire que trois joueurs soient connectés simultanément. De plus, nous devons analyser les logs de chaque machine afin de comparer les comportements et comprendre l'origine des dysfonctionnements. Cette phase d'investigation nous a demandé beaucoup de rigueur et de coordination.

- **Marcelin Ray :**

Je n'ai pas de point précis sur lequel je suis plus fier que d'autres. Je suis très content d'avoir réussi, au fur et à mesure des itérations, à travailler avec mon binôme pour réaliser les deux moteurs graphiques (Raycasting et BSP). Je suis très fier des résultats que nous avons obtenus progressivement.

L'utilisation du BSP et la compréhension de son fonctionnement concret ont été particulièrement enrichissantes. Je suis très heureux de maintenant maîtriser ces mécanismes en détail, et surtout de comprendre par moi-même le fonctionnement du

moteur graphique si révolutionnaire de DOOM, ainsi que celui de Wolfenstein 3D, même si ce dernier implique des thématiques techniques et mathématiques moins sophistiquées en termes d'optimisation.

- **Ilyès Lounici :**

Ma principale fierté dans ce projet est d'avoir réussi à implémenter les Steering Behaviors. Je ne voulais pas que les ennemis aient des déplacements robotiques et prévisibles, il fallait donc trouver une solution pour rendre leurs mouvements plus fluides et naturels. Pour y arriver, j'ai dû me plonger dans la lecture du livre *AI for Games* de Ian Milligton, qui a été indispensable pour comprendre comment utiliser des forces vectorielles afin de simuler ces comportements.

Cependant, passer de la théorie à la pratique a été difficile. Comme nous n'utilisons pas de moteur de jeu existant, j'ai dû traduire ces concepts mathématiques complexes directement en code Java. Malgré la difficulté de cette tâche, j'ai réussi à obtenir un résultat très satisfaisant. Voir les monstres se déplacer aujourd'hui de manière crédible est gratifiant, car cela prouve que j'ai réussi à maîtriser un sujet technique que je ne connaissais pas au départ.

- **Cyprian Chailan :**

Je suis assez fier du travail qu'on a réalisé avec Marcelin, notamment sur la structure BSP et tout son fonctionnement. C'était une partie assez technique, mais justement, j'ai beaucoup aimé travailler dessus parce que ça m'a permis d'apprendre énormément de choses et de mieux comprendre certains concepts. En ce qui concerne le travail d'équipe, je trouve qu'on s'est bien débrouillés. On a su se répartir les tâches de manière efficace, et le fait de travailler en binôme au sein du groupe nous a permis de se fixer des tâches réalistes et atteignables.

Ce projet m'a aussi permis d'apprécier davantage le travail en équipe. C'est une compétence que je pense devoir encore développer, et cette expérience m'a montré que collaborer, communiquer et avancer ensemble peut vraiment être un plus. Au fil du projet, on a développé une bonne entente, chacun travaille sérieusement et il y a une vraie cohésion d'équipe.

7. Objectifs à atteindre pour la fin du projet

Le projet aborde désormais une phase critique de convergence où, bien que les fondations technologiques majeures telles que le réseau, le rendu BSP et l'intelligence artificielle soient codées et fonctionnelles, elles opèrent encore de façon modulaire et relativement indépendante. C'est pourquoi l'objectif stratégique primordial pour la fin de l'année universitaire réside dans la fusion systémique des branches, souvent désignée comme le

"Merge" critique. Ce point constitue le principal verrou technique à lever, car il nécessite d'intégrer la logique comportementale multijoueur et la simulation physique, initialement développées sur l'architecture discrète, au sein de l'espace mathématique continu généré par la branche de rendu. Concrètement, cela implique une révision en profondeur de la détection des collisions afin de permettre aux avatars réseau et aux monstres de se mouvoir sans heurts dans l'arborescence polygonale du BSP.

Parallèlement à cette unification technique, le moteur doit connaître une évolution esthétique significative par l'implémentation du plaquage de textures (Texture Mapping). Le moteur BSP ne restituant actuellement que des primitives unies ou des surfaces en fil de fer, le passage au texturage est indispensable. Cette tâche requiert l'association de coordonnées mathématiques interpolées (UV mapping) aux polygones subdivisés par l'arbre BSP, ainsi que l'application d'une correction de perspective inverse lors du tracé des colonnes de pixels, ceci afin d'éviter les distorsions visuelles des motifs sur les murs fuyants.

Enfin, pour compléter ces avancées, l'effort se portera sur la sophistication décisionnelle de l'intelligence artificielle. Alors que les algorithmes de navigation actuels (A^* , RRT^*) assurent les déplacements d'un point à un autre, la logique de haut niveau doit évoluer pour déterminer quand attaquer, fuir ou patrouiller. L'objectif est donc d'abandonner les machines à états finis (FSM), jugées trop rigides, au profit des Arbres de Comportements (Behavior Trees). Cette architecture, standardisée par des jeux modernes comme la série Halo, utilise des nœuds de séquence, de sélection et des décorateurs conditionnels pour offrir une structure modulaire capable de modéliser des tactiques de combat et de traque complexes de manière lisible et évolutive.

8. Planning pour les trois itérations restantes

Pour les 3 prochaines itérations, on prévoit de finir BSP. Ce qui implique d'ajouter l'affichage des textures aux murs. Ajouter l'affichage des sprites dans le rendu d'une frame et de créer une méthode capable de rendre les murs/sprites visibles. (utile pour optimiser le calcul des collisions, ainsi que de permettre la détection des joueurs par les monstres).

On prévoit aussi de mettre en place l'automate qui va permettre de créer le comportement des monstres. On va compléter cet automate avec des sens comme la vue et l'ouïe. Afin de complexifier notre jeu, nous allons tenter de mettre un système de groupe afin que plusieurs ennemis puissent attaquer un joueur en groupe. Le défi sera ensuite de le relier au réseau afin que les joueurs puissent le voir. On risque de devoir optimiser le comportement de groupe afin de ne pas saturer les réseaux.

9. Conclusion

Au terme de ce rapport d'avancement de mi-parcours, l'analyse approfondie des travaux réalisés démontre que le développement du projet tutoré *DOOM Like* progresse de manière particulièrement soutenue, respectant rigoureusement les jalons technologiques planifiés en décembre.

L'ambition initiale, qui consistait à écarter les facilités offertes par les moteurs de jeu contemporains pour reprogrammer intégralement les concepts fondamentaux de la 3D temps réel, porte ses fruits pédagogiques. L'implémentation réussie de l'arbre de partitionnement binaire de l'espace pour le tri géométrique, le développement de protocoles de communication asynchrones en Peer-to-Peer reposant sur de simples Sockets, ou encore l'application des mathématiques stochastiques à l'intelligence artificielle via les variantes RRT*, dotent les membres du groupe d'une compréhension inestimable de l'ingénierie logicielle bas niveau. Les verrous technologiques primaires, qu'ils concernent l'impact désastreux des imprécisions en virgule flottante sur la résolution des hyperplans, ou la résolution des accès concurrents dans les architectures réseau distribuées, ont été méthodiquement diagnostiqués, analysés et contournés.

Bien que le défi d'ingénierie principal, à savoir la fusion systémique entre le moteur graphique continu et la logique réseau, reste à accomplir lors de la prochaine itération, la stricte découpe fonctionnelle et modulaire de l'architecture logicielle actuelle (UML, usage de patrons de conception) isole les composants et limite les risques de régression. Les prochaines semaines de développement constitueront un tournant décisif ; le succès de l'intégration de ces différentes briques technologiques garantira la transformation de ces prouesses mathématiques et algorithmiques en une expérience vidéoludique interactive, fluide et techniquement aboutie.